MACSE502: Programming for Data Science - Master Comprehensive Notes for Final Assessment Test (FAT)

Course Code: MACSE502 | **Course Title:** Programming for Data Science | **L-T-P-C:** 3-0-2-4
Target: Final Assessment Test (FAT), Continuous Assessments (CAT 1 & CAT 2), and Big Data Systems Architecture
Primary Textbooks & References:

1. Alvaro Fuentes, *Become a Python Data Analyst*, Packt Publishing, 2018.
2. Bharti Motwani, *Data Analytics using Python*, Wiley, 2020.
3. Jules S. Damji, Brooke Wenig, Tathagata Das, and Denny Lee, *Learning Spark: Lightning-Fast Data Analytics*, 2nd ed., O'Reilly, 2020.
4. Wes McKinney, *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*, 3rd ed., O'Reilly, 2022.
5. E. Tome, R. Bhattacharjee, and D. Radford, *Data Engineering with Scala and Spark*, Packt Publishing, 2024.
6. Jean-Georges Perrin, *Spark in Action*, 2nd ed., Manning Publications, 2020.

Detailed Table of Contents

1. Module 1: Python Core, Data Structures & Object-Oriented Architecture
2. 1.1 Python Built-In Data Structures: Lists, Tuples, Dictionaries, Sets
3. 1.2 User-Defined Data Structures: Stacks, Queues, Priority Queues
4. 1.3 String Processing, Regex & Advanced Exception Handling
5. 1.4 Object-Oriented Programming: Encapsulation, Mangling, Class/Static Methods
6. 1.5 REST APIs, HTTP Requests & File Handling (JSON/CSV)
7. 1.6 NLP Foundations with NLTK & ChatterBot Corpus Architecture
8. Module 2: Python Libraries for Data Science
9. 2.1 NumPy Foundations: `ndarray`, Broadcasting Rules & Vectorization
10. 2.2 Pandas Deep Dive: Series, DataFrames, Missing Value Management
11. 2.3 Advanced Pandas Operations: Filtering, `.loc` vs `.iloc`, String Accessors
12. 2.4 DataFrame Merging: `pd.merge()`, `pd.concat()`, `join()` & Split-Apply-Combine
13. 2.5 Web Scraping: BeautifulSoup (DOM Parser) vs. Scrapy (Twisted Asynchronous Engine)
14. 2.6 Visualization Ecosystem: Matplotlib, Seaborn, Bokeh, Plotly, Folium
15. 2.7 ML & Deep Learning Frameworks: Scikit-Learn, Keras, PyTorch, TensorFlow, XGBoost
16. Module 3: Scala Functional & Object-Oriented Programming
17. 3.1 Scala Fundamentals: `val` vs `var`, Type Inference, Functions & REPL
18. 3.2 Scala Collections: Arrays, Lists (Cons `::`), Tuples, Sets, Maps
19. 3.3 Functional Combinators: `map`, `flatMap`, `filter`, `foldLeft`, `reduce`
20. 3.4 Object-Oriented Constructs: Classes, Constructors (Primary & Auxiliary), Singletons
21. 3.5 Companion Objects & The `apply` Factory Pattern
22. 3.6 Case Classes: Built-In Immutability, Copying & Pattern Matching
23. 3.7 Abstract Classes vs. Traits: Multiple Inheritance via Trait Mixins
24. Module 4: Scala Scientific Libraries & Apache Spark Architecture
25. 4.1 Scala Scientific Computing: Breeze, Saddle & Spire Libraries
26. 4.2 Functional Error Handling: `Option[T]`, `Try[T]`, `Either[L, R]`

27. 4.3 Apache Spark Architecture: Driver, Cluster Manager, Executors, Tasks
28. 4.4 Resilient Distributed Datasets (RDD): Partitioned, Immutable, Lazy DAG Lineage
29. 4.5 RDD Operations: Transformations (Lazy) vs. Actions (Eager)
30. 4.6 Spark Ecosystem: Spark Core, Spark SQL, Structured Streaming, MLlib, GraphX
31. Module 5: Programming Language Selection Matrix
32. 5.1 The 13 Selection Criteria for Enterprise Data Platforms
33. 5.2 Comparative Analysis: Python vs. Scala vs. Java vs. R
34. Module 6: Contemporary Issues & High-Yield FAT Review
35. 6.1 Top 10 High-Frequency Theory Questions for FAT
36. 6.2 Python vs. Scala Syntax & Paradigm Cheat Sheet

Module 1: Python Core, Data Structures & Object-Oriented Architecture

1.1 Python Built-In Data Structures: Deep Comparison

Structure	Mutability	Ordering	Duplicates?	Internal Implementation	Complexity (Access / Search)
List (`[]`)	Mutable	Ordered	Allowed	Dynamic Array of object pointers	Access: $O(1)$, Search: $O(n)$
Tuple (`()`)	Immutable	Ordered	Allowed	Fixed-size contiguous array	Access: $O(1)$, Search: $O(n)$
Dict (`{}`)	Mutable	Insertion-order	Unique Keys	Sparse Hash Table with compact index	Key Lookup: $O(1)$ average
Set (`set()`)	Mutable	Unordered	Unique Only	Hash Table (Keys only, dummy values)	Membership `in` : $O(1)$ average

---

1.2 User-Defined Data Structures in Python

- **Stack:** Use `collections.deque` (`append()` , `pop()`) for thread-safe, memory-efficient $O(1)$ operations (avoiding list reallocation).
- **Queue:** Use `collections.deque` (`append()` , `popleft()`) for strict $O(1)$ FIFO performance.
- **Priority Queue:** Use the standard library `heapq` module: `python import heapq pq = [] heapq.heappush(pq, (priority, item)) priority, item = heapq.heappop(pq)`

1.3 Object-Oriented Programming (OOP) in Python

```
class PipelineComponent:
    def __init__(self, name: str):
        self.name = name            # Public attribute
        self._stage = 1             # Protected by convention
        self.__secret_key = "token" # Private (Name Mangling: _PipelineComponent__secret_key)

    def execute(self):
        raise NotImplementedError("Subclasses must implement execute()")

    @classmethod
    def from_dict(cls, data: dict):
        return cls(data["name"])

    @staticmethod
    def validate_name(name: str) -> bool:
        return len(name) > 0 and name.isalnum()
```

Module 2: Python Libraries for Data Science

2.1 NumPy Broadcasting Rules

Two arrays can broadcast together if for each trailing dimension: 1. The dimensions are equal, OR 2. One of the dimensions is 1. If neither holds, a `ValueError: operands could not be broadcast together` is raised.

2.2 Pandas Deep Dive: Merging & Aggregation

- **`pd.merge()` vs. `pd.concat()`:**
- `pd.merge(df1, df2, on='key', how='inner'|'left'|'right'|'outer')`: Relational database joins matching on common key columns.
- `pd.concat([df1, df2], axis=0)`: Stacks DataFrames vertically (unions rows); `axis=1` concatenates horizontally (joins columns).
- **Split-Apply-Combine Workflow:** `python df.groupby(['Category', 'Region']).agg( total_sales=('Sales', 'sum'), avg_profit=('Profit', 'mean'), max_margin=('Margin', 'max') ).reset_index()`

2.3 Web Scraping: BeautifulSoup vs. Scrapy

Feature	BeautifulSoup (BS4)	Scrapy Framework
Architecture	Synchronous HTML/XML DOM parser library	Asynchronous event-driven framework (Twisted)
Concurrency	Single-threaded by default	Highly concurrent non-blocking HTTP requests
Pipelines	Custom user scripting required	Built-in Item Pipelines, Exporters & Feed storage
Throttling	Manual `time.sleep()`	Built-in AutoThrottle and politeness delay rules
Primary Use	Quick scripts, small ad-hoc pages	Large-scale automated enterprise web crawling

Module 3: Scala Functional & Object-Oriented Programming

3.1 Scala Fundamentals

- **Immutability First:** `val x = 10` declares an immutable constant; `var y = 20` declares a mutable variable.
- **Pure Functions:** `scala def multiply(a: Int, b: Int): Int = a * b`

3.2 Functional Combinators

- **`map`:** Transforms every element: `List(1, 2, 3).map(_ * 2) // List(2, 4, 6)`
- **`filter`:** Selects elements satisfying predicate: `List(1, 2, 3, 4).filter(_ % 2 == 0) // List(2, 4)`
- **`flatMap`:** Maps and flattens nested collections: `List("hello", "world").flatMap(_.toList)`
- **`foldLeft`:** Accumulates from left with initial accumulator value: `scala val sum = List(1, 2, 3, 4).foldLeft(0)((acc, x) => acc + x) // 10`

3.3 Case Classes & Pattern Matching

Case classes are immutable by default and automatically generate constructor parameters as public fields, structural `equals`, `hashCode`, `toString`, and a `copy` method:

```
case class Transaction(id: String, amount: Double, status: String)

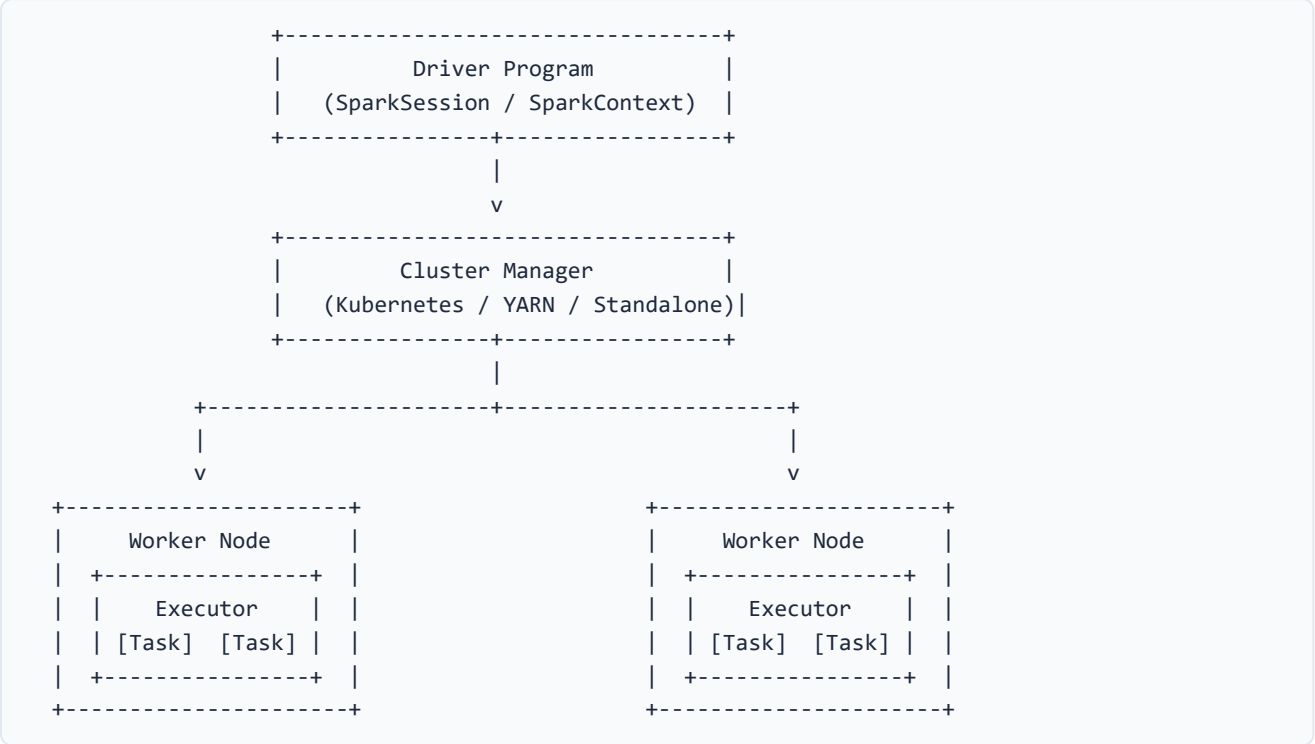
def processTransaction(tx: Transaction): String = tx match {
  case Transaction(id, amt, "SUCCESS") if amt > 10000 => s"Large transaction $id logged"
  case Transaction(id, _, "SUCCESS")                  => s"Standard transaction $id approved"
  case Transaction(id, _, "FAILED")                   => s"Alert: Transaction $id failed"
  case _                                              => "Unknown transaction state"
}
```

3.4 Companion Objects & Traits

- **Companion Object:** An `object` sharing the name of a class in the same file. Has access to private members of the class and defines the `apply` factory method, enabling object instantiation without `new`: `scala class Student(val name: String, val id: Int) object Student { def apply(name: String, id: Int): Student = new Student(name, id) } val s = Student("Rohan", 101) // Uses apply`
- **Traits & Mixins:** Equivalent to Java interfaces but can contain concrete method implementations. A class inherits multiple traits via mixin composition (`class Worker extends Person with Logger with Serializer`).

Module 4: Scala Scientific Libraries & Apache Spark Architecture

4.1 Apache Spark Cluster Architecture



- **Driver Program:** Coordinates job execution; converts high-level code into a **Directed Acyclic Graph (DAG)** of execution stages.
- **Cluster Manager:** Schedules and allocates cluster resources.
- **Executors:** Worker processes running on nodes that execute tasks and cache partitioned RDD data in RAM/disk.

4.2 Resilient Distributed Datasets (RDD)

- **Four Fundamental Characteristics:**
- **Resilient:** Fault-tolerant via DAG Lineage (if a partition is lost, it is recomputed on-the-fly).
- **Distributed:** Data is split into partitions across cluster worker nodes.
- **Dataset:** Holds collections of typed data records.
- **Immutable:** Once created, cannot be mutated; transformed into new RDDs.
- **Transformations vs. Actions:**
- **Transformations (Lazy):** Build DAG lineage graph without executing computation (`map` , `filter` , `flatMap` , `reduceByKey` , `groupByKey`).
- **Actions (Eager):** Trigger physical cluster computation and return values to the driver or write to storage (`count()` , `collect()` , `take()` , `saveAsTextFile()`).

Module 5: Programming Language Selection Matrix

5.1 The 13 Selection Criteria

When selecting languages for enterprise data architectures, evaluate across these 13 criteria: 1. **Size of Deployment (Data, Load & Concurrency):** High distributed throughput favors JVM/Scala/Go; exploratory analytics favors Python. 2. **Security & Cryptography Support:** Enterprise RBAC, memory safety, sandbox execution. 3. **Skill Set & Engineering Availability:** Python has largest data science talent pool; Scala has specialized data engineering focus. 4. **Targeted Platform:** Cloud-native Kubernetes, JVM clusters, embedded IoT edge devices. 5. **Elasticity of Language:** Ability to handle both rapid scripting and massive scale. 6. **Time to Production (Velocity):** Python enables fastest prototyping and experiment iterations. 7. **Execution Performance & Latency:** Compiled JVM bytecode (Scala/Java) outperforms interpreted Python for CPU-bound computations. 8. **Ecosystem & Community Support:** Richness of open-source packages (PyPI vs. Maven). 9. **Purpose:** Heavy distributed big data infrastructure (Scala/Spark) vs. Applied ML/Deep Learning (Python/PyTorch). 10. **Programmer Experience & Developer Ergonomics:** Syntactic conciseness, dynamic vs static typing. 11. **Ease of Maintenance & Refactoring:** Static typing in Scala prevents whole classes of runtime type errors during large refactors. 12. **Computational & Memory Efficiency:** Garbage collection tuning, RAM footprint. 13. **Tooling, IDE & Diagnostics:** Richness of debuggers, profilers, linters (PyCharm, IntelliJ IDEA).

Module 6: Contemporary Issues & High-Yield FAT Review

6.1 Top 10 High-Frequency FAT Exam Questions

1. **Explain the Split-Apply-Combine Strategy in Pandas:** Write code demonstrating `groupby` with multiple aggregation functions.
2. **Compare BeautifulSoup and Scrapy:** Detail why Scrapy's Twisted event loop delivers higher web scraping throughput.
3. **What is an RDD in Apache Spark?** Define its four properties and explain how DAG lineage achieves fault tolerance without data replication.
4. **Transformations vs. Actions in Spark:** List 3 transformations and 3 actions, explaining why lazy evaluation optimizes execution plans.
5. **Scala Case Classes:** Detail four built-in capabilities that distinguish case classes from standard classes.
6. **Companion Objects & The `apply` Method:** Explain how companion objects eliminate the `new` keyword in Scala.
7. **Evaluate Python vs. Scala across the 13 Selection Criteria:** Provide an architectural justification for when to use each.
8. **NumPy Broadcasting Rules:** State the mathematical conditions under which two multi-dimensional arrays can broadcast.
9. **Scala Trait Mixins:** Contrast traits with abstract classes and explain how Scala resolves multiple inheritance diamond issues via linearization.
10. **Spark Catalyst Optimizer:** Explain how Spark SQL optimizes queries through logical optimization, physical planning, and code generation.